Caso Práctico Neurona de McCulloch y Pitts

October 13, 2026

Caso Práctico: Neurona de McCulloch y Pitts

12 de Octubre de 2026

Aplicación de la MP Neurona a un caso práctico real

Kevin Xavier Suasnavas Villarreal

Link a mi GitHub

https://github.com/kevinsuas433/Machine2026.git

---

1 Caso Práctico: Neurona de McCulloch y Pitts

1.1 2. Aplicando la MP Neurona a un caso práctico real

1.1.1 2.1. Conjunto de datos

En esta práctica se utiliza el conjunto de datos Breast Cancer Wisconsin, disponible en la biblioteca `scikit-learn`.

Las características de entrada se calculan a partir de imágenes digitalizadas de muestras obtenidas mediante aspiración con aguja fina (FNA). Estas características describen propiedades de los núcleos celulares presentes en las imágenes.

El conjunto de datos permite trabajar con dos clases: tumores benignos y malignos. Primero se revisará la estructura de los datos y después se prepararán las características para utilizarlas con una neurona MP.

[1]:
```
# Importamos el conjunto de datos de cáncer de mama de sklearn.
from sklearn.datasets import load_breast_cancer

# Cargamos la información en una variable.
cancer_mama = load_breast_cancer()

# Separamos las características de entrada y las etiquetas de salida.
X = cancer_mama.data
Y = cancer_mama.target
```

[2]:
```
# Mostramos los atributos que contiene el conjunto de datos.
dir(cancer_mama)
```

[2]:
```
['DESCR',
 'data',
 'data_module',
 'feature_names',
 'filename',
 'frame',
 'target',
 'target_names']
```

[3]:
```
# Visualizamos las características numéricas de las muestras.
print(X)
```

```
[[1.799e+01 1.038e+01 1.228e+02 … 2.654e-01 4.601e-01 1.189e-01]
 [2.057e+01 1.777e+01 1.329e+02 … 1.860e-01 2.750e-01 8.902e-02]
 [1.969e+01 2.125e+01 1.300e+02 … 2.430e-01 3.613e-01 8.758e-02]
 …
 [1.660e+01 2.808e+01 1.083e+02 … 1.418e-01 2.218e-01 7.820e-02]
 [2.060e+01 2.933e+01 1.401e+02 … 2.650e-01 4.087e-01 1.240e-01]
 [7.760e+00 2.454e+01 4.792e+01 … 0.000e+00 2.871e-01 7.039e-02]]
```

1.2 Visualización del conjunto de datos

Para facilitar la interpretación de la información, convertimos las características en un DataFrame de pandas. Cada columna representa una característica de las muestras y cada fila corresponde a un registro.

Después observamos las etiquetas de salida, que indican la clase asignada a cada muestra.

[4]:
```
# Importamos pandas para organizar los datos en una tabla.
import pandas as pd

# Creamos el DataFrame utilizando los nombres originales
# de las características del conjunto de datos.
df = pd.DataFrame(X, columns=cancer_mama.feature_names)

# Mostramos la tabla para revisar su contenido.
df
```

[4]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	
..	…	…	…	…	…	
564	21.56	22.39	142.00	1479.0	0.11100	

```
565        20.13         28.25          131.20     1261.0          0.09780
566        16.60         28.08          108.30      858.1          0.08455
567        20.60         29.33          140.10     1265.0          0.11780
568         7.76         24.54           47.92      181.0          0.05263

     mean compactness  mean concavity  mean concave points  mean symmetry  \
0             0.27760         0.30010              0.14710         0.2419
1             0.07864         0.08690              0.07017         0.1812
2             0.15990         0.19740              0.12790         0.2069
3             0.28390         0.24140              0.10520         0.2597
4             0.13280         0.19800              0.10430         0.1809
..                ...             ...                  ...            ...
564           0.11590         0.24390              0.13890         0.1726
565           0.10340         0.14400              0.09791         0.1752
566           0.10230         0.09251              0.05302         0.1590
567           0.27700         0.35140              0.15200         0.2397
568           0.04362         0.00000              0.00000         0.1587

     mean fractal dimension  ...  worst radius  worst texture  \
0                   0.07871  ...        25.380          17.33
1                   0.05667  ...        24.990          23.41
2                   0.05999  ...        23.570          25.53
3                   0.09744  ...        14.910          26.50
4                   0.05883  ...        22.540          16.67
..                      ...  ...           ...            ...
564                 0.05623  ...        25.450          26.40
565                 0.05533  ...        23.690          38.25
566                 0.05648  ...        18.980          34.12
567                 0.07016  ...        25.740          39.42
568                 0.05884  ...         9.456          30.37

     worst perimeter  worst area  worst smoothness  worst compactness  \
0             184.60      2019.0           0.16220            0.66560
1             158.80      1956.0           0.12380            0.18660
2             152.50      1709.0           0.14440            0.42450
3              98.87       567.7           0.20980            0.86630
4             152.20      1575.0           0.13740            0.20500
..               ...         ...               ...                ...
564           166.10      2027.0           0.14100            0.21130
565           155.00      1731.0           0.11660            0.19220
566           126.70      1124.0           0.11390            0.30940
567           184.60      1821.0           0.16500            0.86810
568            59.16       268.6           0.08996            0.06444

     worst concavity  worst concave points  worst symmetry  \
0             0.7119                0.2654          0.4601
1             0.2416                0.1860          0.2750
```

```
2              0.4504                  0.2430           0.3613
3              0.6869                  0.2575           0.6638
4              0.4000                  0.1625           0.2364
..                ...                     ...              ...
564            0.4107                  0.2216           0.2060
565            0.3215                  0.1628           0.2572
566            0.3403                  0.1418           0.2218
567            0.9387                  0.2650           0.4087
568            0.0000                  0.0000           0.2871

     worst fractal dimension
0                    0.11890
1                    0.08902
2                    0.08758
3                    0.17300
4                    0.07678
..                       ...
564                  0.07115
565                  0.06637
566                  0.07820
567                  0.12400
568                  0.07039

[569 rows x 30 columns]
```

[5]:
```
# Mostramos las etiquetas que indican la clase de cada muestra.
print(Y)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 1 0 0
 1 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1 1 0 0 1 1 1 0 0 1 1 1 1 0 1 1 0 1 1
 1 1 1 1 1 1 0 0 0 1 0 0 1 1 1 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 1 1 1 0 1
 1 1 1 1 1 1 1 1 0 1 1 1 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1 0 1 1 0 0 0 1 0
 1 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1 0 0 0 1 0 1 0 1 1 0 1 0 0 0 0 1 1 0 0 1 1
 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 0 1 0 1 1 1 1 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1
 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 1 1 1 0 0 0 1 1
 1 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 0 0 1 0 0
 0 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1
 1 0 1 1 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 0 1 1
 0 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 0 1
 1 1 1 1 1 1 0 1 0 1 1 0 1 1 1 1 1 1 0 0 1 0 1 0 1 1 1 1 1 1 0 1 1 0 1 0 1 0 0
 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 0 0 0 0 0 0 1]
```

1.3 División del conjunto de datos

Dividimos las características y sus etiquetas en conjuntos de entrenamiento y prueba. El primero se utiliza para ajustar el modelo y el segundo para evaluar sus predicciones.

El parámetro `stratify=Y` ayuda a conservar la proporción de las clases en ambos conjuntos. De esta manera, la evaluación representa mejor la distribución original de las etiquetas.

[6]:
```
# Importamos la función para dividir los datos.
from sklearn.model_selection import train_test_split

# Separamos los datos en entrenamiento y prueba,
# conservando la proporción de las clases.
X_train, X_test, y_train, y_test = train_test_split(
    df, Y, stratify=Y
)

# Mostramos el número de registros de cada conjunto.
print("Tamaño de entrenamiento:", len(X_train))
print("Tamaño de pruebas:", len(X_test))
```

```
Tamaño de entrenamiento: 426
Tamaño de pruebas: 143
```

1.4 Implementación de una MP Neurona más avanzada

En esta sección creamos la clase `MpNeuron`, que contiene los métodos necesarios para realizar predicciones y ajustar el umbral de la neurona.

- `__init__`: inicializa el umbral.
- `model`: suma las entradas y compara el resultado con el umbral.
- `predict`: realiza predicciones para varias muestras.
- `fit`: prueba diferentes umbrales y selecciona el que obtiene la mayor exactitud sobre los datos de entrenamiento.

La neurona MP trabaja con entradas binarias, por lo que será necesario transformar las características numéricas antes de entrenarla.

[7]:
```
# Importamos NumPy para trabajar con arreglos.
import numpy as np

# Importamos la función que calcula la exactitud del modelo.
from sklearn.metrics import accuracy_score


class MpNeuron:
    # Inicializamos la neurona sin un umbral definido.
    def __init__(self):
        self.threshold = None
```

```
    # Sumamos las entradas y comparamos el resultado con el umbral.
    def model(self, x):
        # Las entradas son valores binarios: 0 o 1.
        z = sum(x)

        # Devolvemos True si la suma alcanza el umbral.
        return (z >= self.threshold)

    # Realizamos predicciones para un conjunto de muestras.
    def predict(self, X):
        Y = []

        # Recorremos cada muestra de entrada.
        for x in X:
            result = self.model(x)
            Y.append(result)

        # Convertimos las predicciones en un arreglo de NumPy.
        return np.array(Y)

    # Buscamos el umbral que consigue la mejor exactitud.
    def fit(self, X, Y):
        accuracy = {}

        # Probamos los posibles umbrales según el número
        # de características de entrada.
        for th in range(X.shape[1] + 1):
            self.threshold = th

            # Calculamos las predicciones con el umbral actual.
            Y_pred = self.predict(X)

            # Guardamos la exactitud obtenida.
            accuracy[th] = accuracy_score(Y_pred, Y)

        # Seleccionamos el umbral con mayor exactitud.
        self.threshold = max(accuracy, key=accuracy.get)
```

1.4.1 Transformación de los datos a valores binarios

Las características del conjunto de datos contienen valores numéricos continuos. Sin embargo, la neurona MP de esta práctica utiliza entradas binarias.

Para adaptar los datos, utilizamos `pd.cut` con dos intervalos y etiquetas de 1 y 0. También representamos un ejemplo mediante un histograma para observar cómo se distribuyen los valores en los intervalos.

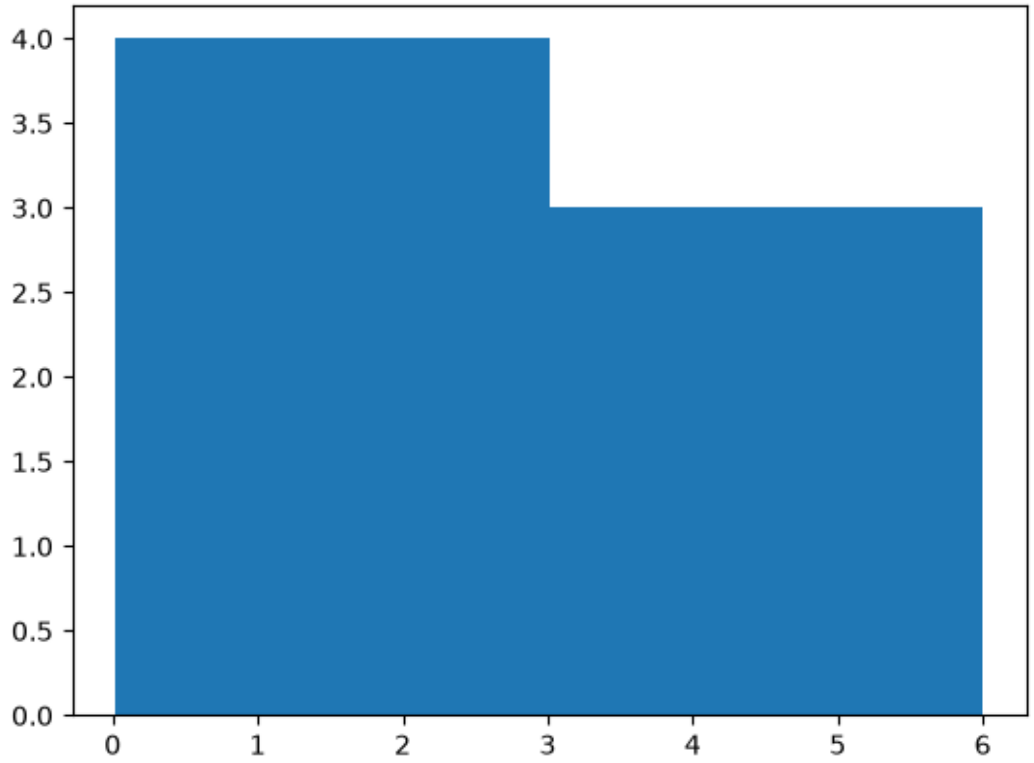
[8]:
```
# Importamos Matplotlib para crear el histograma.
import matplotlib.pyplot as plt

# Dividimos estos valores de ejemplo en dos intervalos
# y asignamos las etiquetas binarias 0 y 1.
valores = [0.04, 0.3, 4, 5, 6, 0.02, 0.6]

print(pd.cut(valores, bins=2, labels=[0, 1]))

# Representamos la distribución de los valores.
plt.hist(valores, bins=2)
plt.show()
```

```
[0, 0, 1, 1, 1, 0, 0]
Categories (2, int64): [0 < 1]
```



[9]:
```
# Transformamos las características de entrenamiento a valores binarios.
X_train_bin = X_train.apply(
    pd.cut, bins=2, labels=[1, 0]
)

# Aplicamos la misma transformación a los datos de prueba.
```

```
X_test_bin = X_test.apply(
    pd.cut, bins=2, labels=[1, 0]
)

# Mostramos los datos binarizados de entrenamiento.
X_train_bin
```

[9]:

```
     mean radius mean texture mean perimeter mean area mean smoothness  \
289            1            1               1         1                1
282            0            1               0         1                1
25             0            1               0         1                0
298            1            1               1         1                1
317            0            1               0         1                1
..           …            …             …       …              …
518            1            1               1         1                0
353            1            0               1         1                1
0              0            1               0         1                0
501            1            0               1         1                0
415            1            1               1         1                1

     mean compactness mean concavity mean concave points mean symmetry  \
289                 1              1                   1              1
282                 1              1                   1              1
25                  0              0                   0              0
298                 1              1                   1              1
317                 1              1                   1              1
..                …              …                 …            …
518                 1              1                   1              1
353                 1              1                   1              1
0                   0              0                   0              0
501                 1              1                   1              0
415                 1              1                   1              1

     mean fractal dimension  … worst radius worst texture worst perimeter  \
289                       1  …            1             1                1
282                       1  …            0             1                0
25                        0  …            0             1                0
298                       1  …            1             1                1
317                       1  …            0             1                0
..                      …  …          …           …              …
518                       1  …            1             1                1
353                       1  …            1             0                1
0                         0  …            0             1                0
501                       1  …            1             0                1
415                       1  …            1             1                1

     worst area worst smoothness worst compactness worst concavity  \
```

```
289          1                1                1                1
282          1                0                1                1
25           1                0                1                1
298          1                1                1                1
317          1                1                1                1
..         …                …                …                …
518          1                1                1                1
353          1                0                1                1
0            0                0                0                0
501          1                0                1                1
415          1                1                1                1

    worst concave points worst symmetry worst fractal dimension
289                     1              1                       1
282                     0              1                       1
25                      0              1                       1
298                     1              1                       1
317                     0              1                       1
..                    …              …                       …
518                     1              1                       1
353                     0              1                       1
0                       0              0                       1
501                     0              1                       1
415                     1              1                       1

[426 rows x 30 columns]
```

1.5 Entrenamiento de la neurona MP

En esta etapa creamos el modelo y lo entrenamos con las características binarias y sus etiquetas correspondientes.

El método `fit` evalúa distintos valores de umbral y conserva el que logra la mayor exactitud en el conjunto de entrenamiento. Después mostramos el umbral seleccionado.

[10]:
```python
# Creamos una instancia de la neurona MP.
np_neurona = MpNeuron()

# Entrenamos el modelo con los datos binarios.
np_neurona.fit(X_train_bin.to_numpy(), y_train)

# Mostramos el umbral óptimo seleccionado.
np_neurona.threshold
```

[10]: 27

1.6 Predicción y evaluación del modelo

En esta sección usamos la neurona entrenada para generar predicciones sobre el conjunto de prueba.

Después calculamos la exactitud comparando las predicciones con las etiquetas reales. Esta métrica permite observar qué proporción de las muestras fue clasificada correctamente.

[11]:
```
# Realizamos las predicciones con las características binarias de prueba.
Y_pred = np_neurona.predict(X_test_bin.to_numpy())
```

[12]:
```
# Mostramos las predicciones obtenidas por la neurona.
Y_pred
```

[12]:
```
array([ True, False, False, False, False,  True,  True,  True,  True,
        True, False, False,  True, False, False, False, False,  True,
       False,  True, False, False,  True, False, False,  True,  True,
        True,  True, False,  True, False,  True,  True,  True,  True,
        True,  True,  True, False, False,  True, False,  True,  True,
       False,  True,  True, False,  True,  True,  True,  True,  True,
       False,  True,  True,  True,  True,  True,  True, False,  True,
       False, False, False, False, False,  True,  True,  True,  True,
        True,  True,  True,  True,  True, False, False, False, False,
        True, False,  True, False, False,  True, False,  True, False,
        True, False, False, False, False, False, False, False,  True,
        True, False,  True,  True,  True, False,  True,  True,  True,
        True, False,  True, False,  True, False,  True, False, False,
       False,  True, False,  True,  True, False, False, False, False,
        True,  True, False, False, False,  True, False, False,  True,
        True, False, False,  True,  True, False,  True,  True])
```

[13]:
```
# Calculamos la exactitud comparando las clases reales
# con las clases predichas por el modelo.
accuracy_score(y_test, Y_pred)
```

[13]:
```
0.8391608391608392
```

[ ]: